Introduction to PyTorch

STAT3612: Statistical Machine Learning

Yinghao Zhu

HKU

November 2026

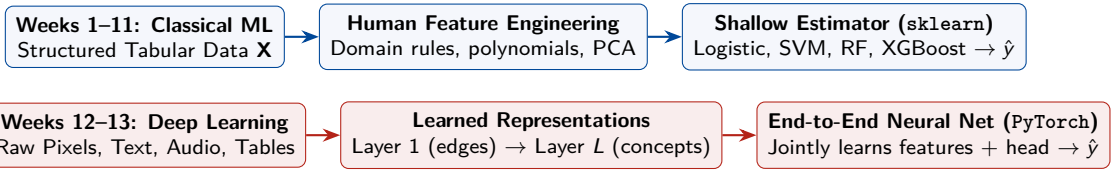
Outline & Companion Notebook

1. Part 1: PyTorch Tensors & Shape Manipulation
2. Part 2: Computational Graphs & Automatic Differentiation (`autograd`)
3. Part 3: Building & Training Neural Networks in PyTorch

Hands-On Companion: `pytorch_intro_tutorial.ipynb`

- **Big-Picture Overview:** Why move from classical `scikit-learn` models to Deep Learning, how Neural Networks generalize Logistic Regression, and the PyTorch mental model.
- **Part 1 (PyTorch Tensors & Manipulation):** Tensor creation, `dtype`/`device`/`requires_grad`, NumPy zero-copy bridge, matrix multiplication `@`, `dim` reductions, `permute`/`squeeze`/`unsqueeze`, and `einops.rearrange`.
- **Part 2 (Computational Graphs & Autograd):** Forward vs. backward propagation, `.backward()`, why `optimizer.zero_grad()` is mandatory, and `torch.no_grad()`.
- **Part 3 (Training Neural Networks):** `Dataset`/`DataLoader`, `nn.Module` MLP & 2D CNN (`LeNet`), `CrossEntropyLoss` + `Adam`, and saving/loading model checkpoints (`state_dict`).

Big-Picture Overview: From Classical ML to Deep Learning



Why Classical ML Hits a Wall on Raw Signals

- In tabular data, each column (x_j = age, BMI) already carries semantic meaning.
- In a raw 224×224 medical image or text document, individual pixels or characters are meaningless in isolation—hand-crafting features fails!

The Deep Learning Paradigm Shift

- **Representation Learning:** Stack simple differentiable layers so the model automatically learns hierarchical features directly from raw data.
- Scales smoothly with massive datasets and GPU compute.

Bridging What You Know: From Logistic Regression to Neural Nets

1. Single Neuron (Week 3)
$a = \sigma(\mathbf{w}^\top \mathbf{x} + b)$
(= Logistic Regression)

→

2. Hidden Layer (MLP)
$\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$
(H parallel feature detectors)

→

3. Deep Architectures
MLP (vectors) · **CNN** (2D grids)
Transformer (token sequences)

Forward Pass: Nested Function Composition

- A single neuron $\sigma(\mathbf{w}^\top \mathbf{x} + b)$ is literally Logistic Regression—restricted to a flat hyperplane.
- Stacking L linear layers interleaved with non-linear activations (ReLU, GELU) bends the feature space to approximate any continuous function:

$$\hat{\mathbf{y}} = f_\theta(\mathbf{x}) = \mathbf{W}^{(L)}\sigma\big(\cdots\sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})\big)$$

Backward Pass: Week 2 Gradient Descent!

- How do we fit millions of weights $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$? Using the exact same Stochastic Gradient Descent from Week 2:

$$\theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}(\hat{\mathbf{y}}, y)$$

- **Backpropagation** is simply the calculus Chain Rule applied backward from $\mathcal{L}$ to layer 1.

What Is PyTorch & How Does It Differ from `scikit-learn`?

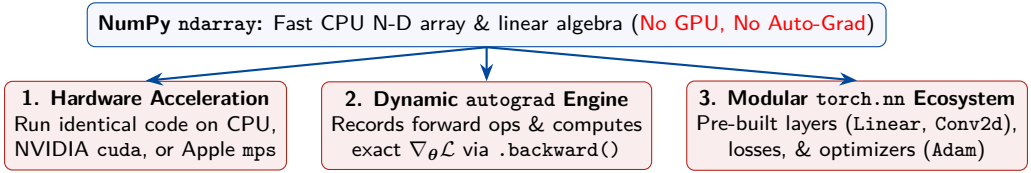
Background: Created by Meta AI (FAIR) and now part of the Linux Foundation, PyTorch is the dominant deep learning framework in both academic research ($> 85\%$ of papers) and frontier AI industry (Hugging Face, Llama, DeepSeek, GPT).

Dimension	`scikit-learn` **(Weeks 1–11)**	`PyTorch` **(Weeks 12–13)**
Philosophy	**Closed-box:** `model.fit(X, y)`	**White-box Lego blocks:** custom graph
Data Type	CPU `numpy.ndarray` / `DataFrame`	CPU / GPU `torch.Tensor` + autograd
Optimization	Built-in solver (L-BFGS, tree split)	Explicit loop (`zero_grad` $\rightarrow$ `backward` $\rightarrow$ `step`)
Modalities	Tabular feature matrices (n, d)	Tables (B, d), seqs (B, T, D), images (B, C, H, W)

The Three Modular Pillars of PyTorch (Our Roadmap for Today)

1. **Part 1 (`torch.Tensor`):** Multi-dimensional arrays that run on GPUs and represent both data batches and model weights.
2. **Part 2 (`torch.autograd`):** Automatic differentiation engine that records forward math and computes $\nabla_{\theta}\mathcal{L}$ via `.backward()`.
3. **Part 3 (`torch.nn` & `torch.optim`):** Pre-built layers (`Linear`, `Conv2d`), loss functions, `DataLoader` batching, and optimizers (`Adam`).

Part 1 Bridge: Why `torch.Tensor` Instead of NumPy `ndarray`?



What Is a `torch.Tensor`?

A multi-dimensional array (0D scalar $\rightarrow$ 1D vector $\rightarrow$ 2D matrix $\rightarrow$ 3D sequence (B, T, D) $\rightarrow$ 4D image batch (B, C, H, W)) that lives on a target `device` and optionally tracks a computational graph for automatic differentiation.

Creating Tensors & The Four Core Tensor Attributes

```python
import torch
# 1.  From Python lists or factory functions
a = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
z = torch.zeros((2, 3), dtype=torch.float32)
r = torch.randn((64, 10))  # standard normal N(0,1)
u = torch.arange(0, 10, step=2)
# 2.  Match shape & device of existing tensor x
like_x = torch.zeros_like(a)
print(a.shape, a.dtype, a.device, a.requires_grad)
```

Four Attributes to Check When Debugging:

- `x.shape`: Tuple of dimensions, e.g., `torch.Size([64, 10])`.
- `x.dtype`: Use `torch.float32` for weights/features and `torch.int64` (`long`) for class labels / token IDs!
- `x.device`: cpu or `cuda:0` (all tensors in an operation must share the same device).
- `x.requires_grad`: `True` if PyTorch should compute $\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$.

Device Management, Seeding & The NumPy Zero-Copy Bridge

```
# 1. Reproducible random seed
torch.manual_seed(42)
# 2. Device-agnostic code pattern
device = "cuda" if torch.cuda.is_available() else
"cpu"
x = torch.randn(4, 3, device=device)
model = model.to(device)
# 3. NumPy <-> PyTorch bridge
arr = np.ones((2, 3), dtype=np.float32)
t = torch.from_numpy(arr)  # shares CPU memory!
arr_back = x.detach().cpu().numpy()
```

Key Practical Rules:

- **Shared Memory on CPU:** `torch.from_numpy(arr)` and `t.numpy()` share underlying RAM without copying!
- **Safe Conversion to Matplotlib / Sklearn:** If a tensor is on GPU or tracks gradients, always call `x.detach().cpu().numpy()` before plotting or passing to `scikit-learn` metrics.

Elementwise Operations vs. Matrix Multiplication (@)

```
A = torch.randn(2, 3)
B = torch.randn(2, 3)
C = torch.randn(3, 4)
# 1. Elementwise (Hadamard) product: (2, 3)
elem = A * B     # same as torch.mul(A, B)
# 2. Matrix multiplication: (2,3) @ (3,4) -> (2,4)
out = A @ C      # same as torch.matmul(A, C)
# 3. Batched 3D matmul: (B,N,K) @ (B,K,M) ->
(B,N,M)
Q = torch.randn(32, 10, 64)
K_t = torch.randn(32, 64, 10)
scores = Q @ K_t   # shape: (32, 10, 10)
```

Why `@` (`torch.matmul`) Is Essential:

- `A * B`: Multiplies entry-by-entry $A_{ij}B_{ij}$ (requires matching or broadcastable shapes).
- `A @ C`: Contracts inner dimension K: $(N, K) \times (K, M) \to (N, M)$.
- **Batched `@`:** Automatically broadcasts over leading batch/head dimensions (B, H, T, d_k)—the core operation of neural layers and Transformer attention!

Visual Intuition: Tensor Reductions Along `dim`

```
x = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])  # (2, 3)
# Collapse dim=0 (rows) -> shape (3,)
c_sum = x.sum(dim=0) # tensor([5., 7., 9.])
# Collapse dim=1 (cols) -> shape (2,)
r_sum = x.sum(dim=1) # tensor([6., 15.])
# Keep collapsed axis of size 1 for
broadcasting
r_keep = x.mean(dim=1, keepdim=True) # (2,
1)
```

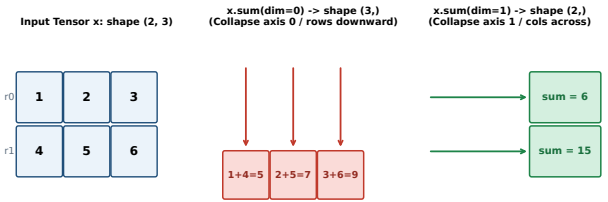


Figure Takeaway: On a $(2, 3)$ matrix, `dim=0` collapses the 2 rows downward into a $(3,)$ vector $[5, 7, 9]$, whereas `dim=1` collapses the 3 columns across into a $(2,)$ vector $[6, 15]$.

- **Mental Rule:** The dimension specified in `dim=k` is the axis that gets collapsed/eliminated!

Reshaping, Indexing & Slicing (reshape vs. view)

```
x = torch.arange(12)      # shape:  (12,)
M = x.reshape(3, 4)       # shape:  (3, 4)
# Use -1 to infer the remaining dimension:
B = torch.randn(32, 1, 28, 28) # (B, C, H, W)
flat = B.reshape(B.size(0), -1) # (32, 784)
# Slicing & Boolean masking
first_two_cols = M[:, :2]    # shape:  (3, 2)
pos_vals = B[B > 0]          # 1D filtered tensor
```

`view` vs. `reshape`:

- **`x.view(new_shape)`:** Zero-copy view, but requires `x` to be contiguous in memory (raises an error right after `transpose`/`permute`).
- **`x.reshape(new_shape)`:** Returns a zero-copy view whenever possible and automatically copies if non-contiguous—prefer `reshape`!
- **Auto-Inference (`-1`):** `B.reshape(B.size(0), -1)` safely flattens any spatial dimensions into a 2D feature matrix (B, D).

Permuting, Squeezing & UnSqueezing Axes

```
# 1. Reorder axes: HWC (plt) -> CHW (PyTorch)
img_hwc = torch.randn(28, 28, 3)
img_chw = img_hwc.permute(2, 0, 1) # (3, 28, 28)
# 2. unsqueeze(dim): insert a size-1 batch axis
single_x = img_chw.unsqueeze(0)  # (1, 3, 28, 28)
# 3. squeeze(dim): remove a size-1 axis
logits = torch.randn(64, 1)
scores = logits.squeeze(1)     # (64,)
```

When You Need Each Operation:

- `permute(d0, d1, ...)`: Reorders multiple axes at once (e.g., converting Matplotlib (H, W, C) images to PyTorch (C, H, W) format).
- `unsqueeze(0)`: Turns a single sample (C, H, W) into a batch of size 1 $(1, C, H, W)$ so `model(single_x)` works!
- `squeeze(1)`: Strips singleton output dimensions $(B, 1) \to (B,)$ to match 1D target vectors.

Self-Documenting Tensor Manipulation with `einops.rearrange`

```python
from einops import rearrange
imgs = torch.randn(32, 1, 8, 8) # (b, c, h, w)
# 1. Flatten spatial axes into feature vector
flat = rearrange(imgs, "b c h w -> b (c h w)")
# shape: (32, 64)
# 2. Split 8x8 image into sixteen 2x2 patches!
patches = rearrange(imgs,
  "b c (h p1) (w p2) -> b (h w) (p1 p2 c)",
  p1=2, p2=2) # shape: (32, 16, 4)
```

- **Why `einops`:** Replaces error-prone chains of `reshape` + `permute` with readable Einstein-style axis equations!

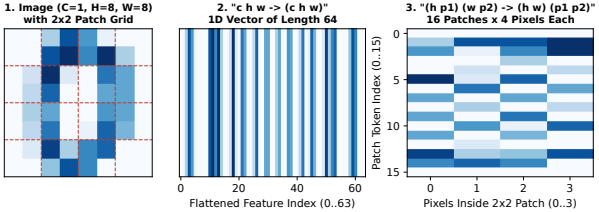
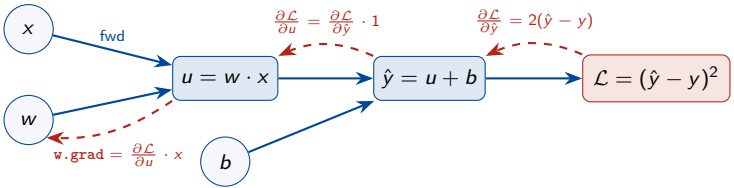


Figure Takeaway: `einops.rearrange` expresses both MLP flattening `"b c h w -> b (c h w)"` and Vision Transformer patchification `"... -> b (h w) (p1 p2 c)"` in one self-documenting line.

Visual Intuition: Dynamic Computational Graphs & Reverse-Mode Autograd



1. Forward Pass (Blue Solid Arrows)

Computes output $\hat{y}$ and scalar loss $\mathcal{L}$ while dynamically recording a directed acyclic graph (`grad_fn`) of every operation applied to tensors with `requires_grad=True`.

2. Backward Pass (`loss.backward()`, Red Dashed)

Applies the reverse-mode chain rule from scalar $\mathcal{L}$ back to every leaf parameter w, b, storing $\frac{\partial \mathcal{L}}{\partial w}$ inside `w.grad`.

Hands-On Autograd: requires_grad=True & .backward()

```
# 1.  Mark leaf tensor to track gradients
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
# 2.  Forward pass:  y = sum(x_i^2 + 3*x_i)
y = (x ** 2 + 3 * x).sum()
print(y.grad_fn)  # <SumBackward0>
# 3.  Backward pass:  dy/dx_i = 2*x_i + 3
y.backward()
print(x.grad)     # tensor([5., 7., 9.])
# Non-scalar vector y_vec requires vector v:
# y_vec.backward(v) computes v^T * Jacobian
```

How `.backward()` Works:

- **Analytical Check:** For $y = \sum_i (x_i^2 + 3x_i)$, the derivative is $\frac{\partial y}{\partial x_i} = 2x_i + 3$. At $\mathbf{x} = [1, 2, 3]$, `x.grad` is $[2(1) + 3, 2(2) + 3, 2(3) + 3] = [5, 7, 9]$.
- **Scalar Loss Rule:** `y.backward()` with no arguments requires `y` to be a 0D scalar (such as mean batch loss).
- **Vector-Jacobian Product:** If $\mathbf{y}$ is a vector, `y.backward(v)` computes $\mathbf{v}^\top \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$.

Critical Pitfall: Why `optimizer.zero_grad()` Is Mandatory

```
x = torch.tensor([2.0], requires_grad=True)
for step in range(3):
  if x.grad is not None:  x.grad.zero_()
  y = x ** 2 + 3 * x
  y.backward() # does x.grad += dy/dx!
  print(x.grad)
```

- **Why PyTorch Accumulates by Default:** Calling `.backward()` executes `param.grad += new_grad` (useful when accumulating gradients over micro-batches).
- **Takeaway:** In a standard training loop, always call `optimizer.zero_grad()` before `loss.backward()`!

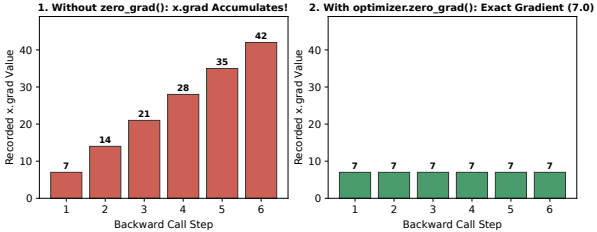


Figure Takeaway: Without `zero_grad()`, repeated `backward()` calls at $x = 2$ accumulate $7 \to 14 \to 21 \to 28$; calling `zero_grad()` resets the buffer so every step uses the true gradient 7.0.

Controlling Gradient Tracking: `torch.no_grad()` & `.detach()`

```
# 1. Context manager for validation / test inference
model.eval()
with torch.no_grad():
  val_logits = model(X_val)
  val_loss = criterion(val_logits, y_val)
# 2. Detach a single tensor from the graph
z = model(X_train)
z_frozen = z.detach()  # requires_grad=False
```

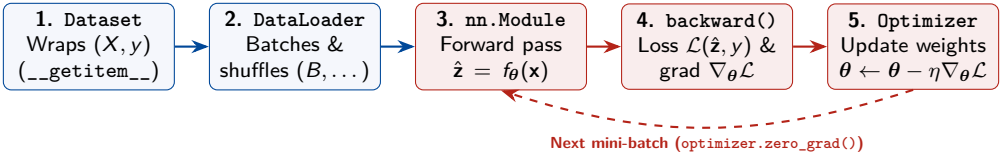
Why Disable Autograd During Inference?

- **Saves Memory:** Forward activations do not need to be cached in RAM/VRAM for a backward pass.
- **Speeds Up Evaluation:** Skips computational graph bookkeeping.
- **`model.eval()` vs. `torch.no_grad()`:**
  - `model.eval()` switches `Dropout` and `BatchNorm` layers to inference mode.
  - `torch.no_grad()` turns off the autograd engine. **Use both together** during validation!

Part 1 & 2 Summary: Tensor & Autograd Cheat Sheet

Task	PyTorch Idiom	Key Note / Pitfall to Avoid
Create float/label tensors	`torch.tensor(x, dtype=float32/int64)`	`CrossEntropyLoss` needs `int64` labels
Move to GPU / CPU	`x.to(device)`; `model.to(device)`	Model & inputs must share `device`
Convert to NumPy safely	`x.detach().cpu().numpy()`	Detach graph + move off GPU first
Matmul vs. Hadamard	`A @ B` vs. `A * B`	`@` contracts axis; `*` is elementwise
Reduce along axis k	`x.sum(dim=k, keepdim=True)`	`dim=k` is the axis that collapses
Reshape / rearrange	`x.reshape(B, -1)` / `rearrange(...)`	Prefer `reshape`/`einops` over `view`
Compute gradients	`opt.zero_grad()`; `loss.backward()`	Always zero gradients before `backward()`!
Validation / test eval	`model.eval()` + `torch.no_grad()`	Disables Dropout noise & graph memory

Quick Design Recap: The 5-Stage PyTorch Training Pipeline



Data Pipeline (Stages 1–2)

Decouples storage/preprocessing (`Dataset`) from mini-batch sampling, shuffling, and parallel prefetching (`DataLoader`).

Optimization Loop (Stages 3–5)

Every epoch iterates over mini-batches: `zero_grad()` → forward `model(xb)` → `criterion` → `backward()` → `optimizer.step()`.

Step 1: Wrapping Data in Dataset & DataLoader

```
from torch.utils.data import (
  Dataset, DataLoader, TensorDataset)
# Option A: Quick wrapper for in-memory tensors
train_ds = TensorDataset(
  torch.tensor(X_tr, dtype=torch.float32),
  torch.tensor(y_tr, dtype=torch.int64))
# Option B: Mini-batch DataLoader
train_loader = DataLoader(train_ds,
  batch_size=64, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=128,
  shuffle=False)
```

Custom `Dataset` Interface:

- Inherit from `torch.utils.data.Dataset` and implement two methods:
  1. `__len__(self)`: returns total sample count n.
  2. `__getitem__(self, idx)`: returns tuple `(x_i, y_i)`.
- **Shuffling Rule:** Set `shuffle=True` on `train_loader` to break sample ordering bias each epoch, and `shuffle=False` on validation/test loaders!

Dataset & Task Roadmap: Handwritten Digit & Image Classification

Benchmark Tasks in Our Notebook:

- **Task 1 (10-Class Digit/Image Classification):** Classify 8×8 grayscale digit images ($d = 64$ pixels, classes 0–9) and 28×28 Fashion-MNIST images.
- **Task 2 (2D Non-Linear Boundary):** Compare a single linear layer `nn.Linear(2, 1)` against a 2-hidden-layer `ReLU` MLP on `make_moons`.
- **Task 3 (2D Spatial CNN vs. Flat MLP):** Upgrade from a flattened MLP to a 2D Convolutional Neural Network (`Conv2d` + `MaxPool2d`).

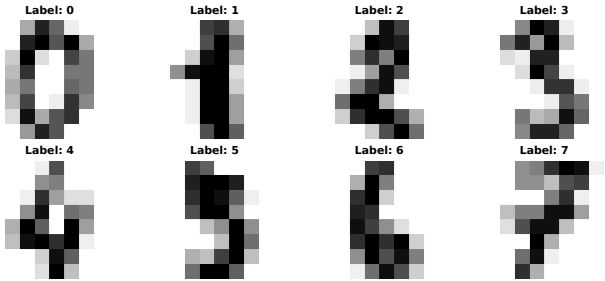


Figure Takeaway: Each 8×8 grayscale image can be viewed either as a flattened 64D feature vector $\mathbf{x} \in \mathbb{R}^{64}$ for an MLP or as a 2D spatial tensor $(1, 8, 8)$ for a CNN.

Step 2: Defining a Multi-Layer Perceptron (nn.Module)

```python
import torch.nn as nn
class MLPClassifier(nn.Module):
  def __init__(self, in_dim=64, num_classes=10):
    super().__init__()
    self.net = nn.Sequential(
      nn.Flatten(),
      nn.Linear(in_dim, 64),
      nn.ReLU(),
      nn.Dropout(p=0.1),
      nn.Linear(64, num_classes)) # raw logits!
  def forward(self, x):
    return self.net(x)
```

Three Rules for `nn.Module`:

1. Always call `super().__init__()` first so PyTorch registers trainable parameters.
2. Define layers in `__init__` and computation in `forward(self, x)`; invoke the model via `model(x)` (never call `model.forward(x)` directly!).
3. **No Final Softmax:** Return raw unnormalized logits from the final `nn.Linear` layer!

Visual Intuition: Why Non-Linear Activations Are Essential

Without Activation $\sigma(\cdot)$, Depth Collapses:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \tilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}}$$

A stack of linear layers without non-linearities is still just a single linear model!

- `Sigmoid` / `Tanh`: Saturate for $|z| > 3$, causing vanishing gradients ($\sigma'(z) \approx 0$) in deep networks.
- `ReLU` ($\max(0, z)$) & `GELU`: Non-saturating for $z > 0$ ($\sigma'(z) = 1$), enabling fast, stable gradient flow.

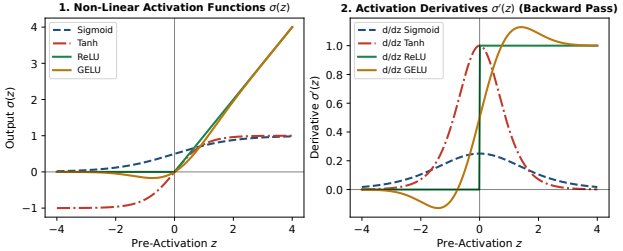


Figure Takeaway: `Sigmoid` and `Tanh` derivatives vanish toward 0 in the tails ($|z| > 3$), whereas `ReLU` and `GELU` maintain unit slope ($\sigma'(z) \approx 1$) for positive activations.

Visual Proof: Single Linear Layer vs. 2-Layer ReLU MLP on make_moons

```
# 1. Single linear layer = Logistic Regression
lin_net = nn.Linear(2, 1)
# 2. 2-hidden-layer ReLU MLP
deep_net = nn.Sequential(
  nn.Linear(2, 32), nn.ReLU(),
  nn.Linear(32, 32), nn.ReLU(),
  nn.Linear(32, 1))
```

- **Universal Approximation:** Stacking `Linear` + `ReLU` layers composes piecewise-linear half-spaces into smooth curved boundaries (86.8% → 97.2%).

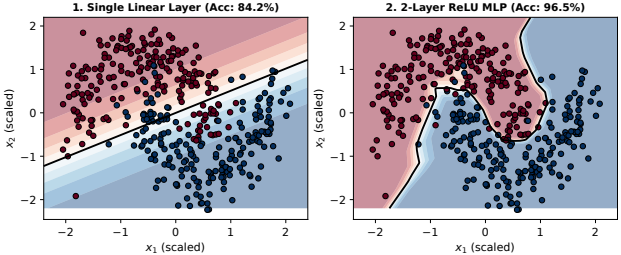


Figure Takeaway: A single `nn.Linear(2, 1)` layer is restricted to a straight hyperplane (86.8%), whereas adding two 32-unit `ReLU` hidden layers bends the boundary around both crescent moons (97.2%).

Step 3: Loss Functions & Optimizers (SGD vs. Adam)

```
# Multi-class loss (fused LogSoftmax + NLLLoss)
criterion = nn.CrossEntropyLoss()
# Binary classification loss on raw logit
bce_loss = nn.BCEWithLogitsLoss()
# Adaptive Moment Estimation optimizer
optimizer = torch.optim.Adam(
  model.parameters(), lr=3e-3,
weight_decay=1e-4)
```

- **Vanilla SGD:** $\theta_{t+1} = \theta_t - \eta \mathbf{g}_t$ (slow along ravines).
- **Adam:** Tracks 1st moment $\mathbf{m}_t$ (momentum) and 2nd moment $\mathbf{v}_t$ (per-parameter adaptive step size)—converges much faster out of the box!

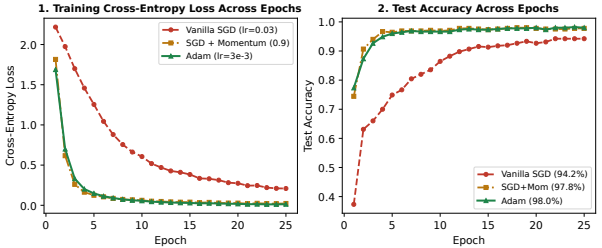


Figure Takeaway: On the 10-class digits MLP, `Adam(lr=3e-3)` and `SGD(momentum=0.9)` minimize cross-entropy loss and reach $> 97.5\%$ test accuracy in a fraction of the epochs required by vanilla `SGD`.

Step 4: The Canonical 5-Step `train_loop` & `eval_loop`

Training Loop (`model.train()`):

```
def train_epoch(model, loader, criterion, opt):
  model.train()   # enable Dropout / BatchNorm
  for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    opt.zero_grad()    # 1.  reset gradients
    logits = model(xb)  # 2.  forward pass
    loss = criterion(logits, yb) # 3.  loss
    loss.backward()    # 4.  backprop
    opt.step()       # 5.  update weights
```

Evaluation Loop (`model.eval()`):

```
def evaluate(model, loader, criterion):
  model.eval()   # disable Dropout noise
  correct, total = 0, 0
  with torch.no_grad():
    for xb, yb in loader:
      logits = model(xb.to(device))
      preds = logits.argmax(dim=1)
      correct += (preds == yb).sum().item()
      total += yb.size(0)
  return correct / total
```

Takeaway: Memorize the 5-Line Heartbeat of PyTorch Training

`opt.zero_grad()` $\longrightarrow$ `logits = model(xb)` $\longrightarrow$ `loss = criterion(logits, yb)` $\longrightarrow$ `loss.backward()` $\longrightarrow$ `opt.step()`

Step 5: Upgrading from Flat MLP to 2D CNN (LeNet) & Diagnostics

```
# Preserve 2D spatial structure (B, 1, H, W)!
cnn = nn.Sequential(
  nn.Conv2d(1, 16, kernel_size=3, padding=1),
  nn.ReLU(),
  nn.MaxPool2d(kernel_size=2), # 8x8 -> 4x4
  nn.Conv2d(16, 32, kernel_size=3, padding=1),
  nn.ReLU(),
  nn.Flatten(),
  nn.Linear(32 * 4 * 4, 10))
```

- **Why CNNs Beat Flat MLPs on Images:** `nn.Conv2d` shares weights across spatial (H, W) patches (translation invariance) instead of flattening 2D neighbors away.

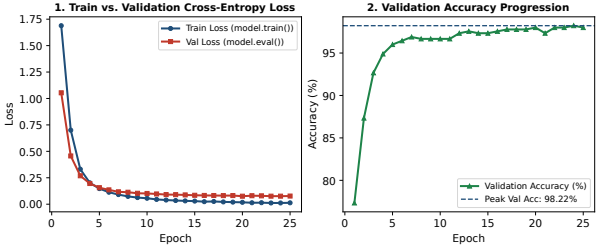


Figure Takeaway: Tracking train loss alongside validation loss and accuracy confirms smooth convergence to 97.78% test accuracy without validation loss divergence.

Saving & Loading Checkpoints (state_dict) & Take-Home Summary

Saving & Restoring Trained Weights:

```
# 1. Save learned parameter dictionary to disk
torch.save(model.state_dict(), "mlp_weights.pt")
# 2. Instantiate architecture & load weights
model_new = MLPClassifier().to(device)
state = torch.load("mlp_weights.pt",
  map_location=device, weights_only=True)
model_new.load_state_dict(state)
model_new.eval()
```

Why Save `state_dict()` Instead of Pickling the Whole Model?

- **Portability:** `model.state_dict()` is a plain Python dictionary mapping each layer name (e.g., `"net.1.weight"`) to its parameter tensor.
- **Device-Agnostic:** Passing `map_location=device` lets you load GPU-trained weights seamlessly onto a CPU machine!

Recommended Open Courses (Full Lecture Videos + Syllabus & Slides)

- **Stanford CS231n (Deep Learning for Computer Vision):** cs231n.stanford.edu — Full YouTube lectures, syllabus, downloadable PDF slides & PyTorch assignments.
- **Andrej Karpathy — Neural Networks: Zero to Hero:** karpathy.ai/zero-to-hero.html — Live-coding autograd (micrograd) to GPT-2 from scratch in PyTorch.